

UMS *JournalService*

This service listens to Kafka brokers, collects the messages on a specific topic and writes them to a central audit log. We are free to choose any platform to build the service, though we chose Java Spring Boot here.

First things first — make sure the *pom.xml* has the following dependencies:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.kafka</groupId>
  <artifactId>spring-kafka</artifactId>
</dependency>
<dependency>
  <groupId>com.fasterxml.jackson.core</groupId>
  <artifactId>jackson-core</artifactId>
</dependency>
<dependency>
  <groupId>com.fasterxml.jackson.core</groupId>
  <artifactId>jackson-databind</artifactId>
</dependency>
```

Build a DTO that captures the user data that is added to the UMS:

```
import java.util.Date;
public class UserRecord {
  private String name;
  private long phone;
  private Date since;
  public UserRecord() {
  }
  public UserRecord(String name, long phone, Date since) {
    this.name = name;
    this.phone = phone;
    this.since = since;
  }
  // getters and services
}
```

Since Kafka only delivers the byte arrays to the consumers, our *JournalService* must deserialize the byte array into the *UserRecord* DTO to get the Java object for further processing.

```
import org.apache.kafka.common.serialization.Deserializer;
import com.fasterxml.jackson.databind.ObjectMapper;
public class UserDeserializer implements
  Deserializer<UserRecord> {
  @Override
```

```
    public UserRecord deserialize(String topic, byte[] data)
  {
    try {
      return new ObjectMapper().readValue(data,
        UserRecord.class);
    } catch (Exception e) {
      e.printStackTrace();
      return null;
    }
  }
}
```

Now, let us build the actual Kafka consumer. Spring Boot offers powerful annotations for Kafka integration. The *JournalHandler* is a Spring Bean that acts as the Kafka listener on a configured topic. Kafka delivers the same message to any number of consumers as long as they are from different consumer groups. To be compatible with this requirement, the *JournalHandler* is configured with a specific group-id as well. Even when multiple instances of the *JournalHandler* are deployed and subscribed to Kafka, none of them receives duplicate messages, thanks to Kafka.

```
import org.springframework.beans.factory.annotation.
  Autowired;
import org.springframework.kafka.annotation.KafkaListener;
import org.springframework.stereotype.Component;
@Component
public class JournalHandler {
  @Autowired
  private JournalAdapter adapter;
  @KafkaListener(topics = "${ums.user.add.topic}", groupId
    = "${ums.user.add.consumer}")
  public void handle(UserRecord user) {
    adapter.record(user.toString());
  }
}
```

Going by the design in Part 3 of this series, we are supposed to use a hypothetical third-party library named *FileJournal* to write the messages into the audit log. Recollect that we employed the *adapter* pattern to build a protection layer. Thus, the *JournalHandler* is wired with *JournalAdapter*. But, since the *FileJournal* is not guaranteed to be a Spring Bean, we need to register it with the Spring context for the adapter to pick it up. The following configuration does this:

```
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import com.glarimy.lib.FileJournal;
@Configuration
public class UserJournalConfiguration {
  @Bean
  public FileJournal getFileJournal() throws Exception {
```